

Comprehensive Placement and Routing Framework with Guaranteed In-Cell Routability for Synthesizing Complementary-FET Cells

Zhengzhe Zheng¹, Yinuo Wu¹, Keyu Peng¹, Chao Wang^{1,2,†}, Ziran Zhu^{1,2,†}

¹National ASIC System Engineering Center, Southeast University, Nanjing, China

²National Center of Technology Innovation for EDA, Nanjing, China

Email: zzzkhorne@gmail.com, {220231843, kypeng, wangc, zrzhu}@seu.edu.cn

Abstract—As the technology node advances beyond 5nm, the conventional FinFET architecture encounters substantial scaling issues. Complementary-FET (CFET) technology, characterized by the vertical stacking of P-FET over N-FET or vice versa, has emerged as a promising solution. However, the inherent characteristics of CFET architecture, particularly the scarcity of routing resources, pose significant obstacles to in-cell routability and layout generation. In this paper, we develop a comprehensive placement and routing framework for synthesizing CFET cells. We first present a partitioning technique followed by a heuristic quality maintenance strategy for large-scale cells to ensure scalability and efficiency. Then, we propose a novel satisfiability modulo theories (SMT)-based placement method that incorporates partial routing to achieve minimum-width placement while ensuring in-cell routability. Particularly, the placement method also determines the pin positions for each net, which simplifies subsequent routing complexity. Finally, we propose a progressive metal routing method to address the challenges of routing resource scarcity and unidirectional routing in CFET technology, which includes a manual-inspired M0 routing followed by an integral linear programming (ILP)-based M1 and M2 routing. Compared with the state-of-the-art CFET cell generators, experimental results show that our algorithm achieves the smallest cell width for all tested cells, with 7 out of 30 cells exhibiting smaller widths. For the remaining 23 cells, which have the same cell width as those in other generators, our algorithm achieves the smallest M2 usage and total metal length.

Index Terms—Standard cell, CFET, placement, routing, routability

I. INTRODUCTION

With the continuous advancement of technology nodes, the conventional FinFET structure encounters scaling-related challenges [15]. Various methods have been proposed to reduce cell size, including fin depopulation technology [2] and nano-sheet FET structure [1, 9]. However, these scaling techniques are constrained by physical limitations, such as lateral spacing requirements. Recently, complementary-FET (CFET) technology [10, 12, 14] has emerged as a promising solution. Compared to conventional FinFET structures, CFET technology effectively reduces area by stacking NMOS and PMOS devices, as illustrated in Figure 1(a).

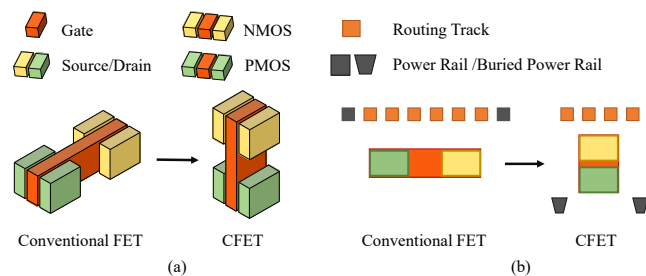


Fig. 1. (a) Example of conventional FET and CFET structures. (b) Cross-sectional view of conventional FET and CFET structures with routing tracks.

While CFET technology offers significant potential for reducing cell area, it also leads to fewer available routing tracks [13], as shown in Figure 1(b). To manage routing within the constrained resources, buried power rails (BPR) are incorporated, allowing power delivery

through a backside network that frees up more routing space [11]. However, the limited routing resources and stringent design rules still pose considerable challenges for CFET in-cell routability. Ensuring in-cell routability during the placement stage is essential for a seamless transition to routing and achieving high-quality layouts.

In recent years, several works [4, 7] have developed innovative methods for CFET layout generation. The method in [4] proposes a satisfiability modulo theories (SMT)-based method for simultaneous placement and routing, enabling it to ensure in-cell routability and achieves the desired CFET layouts. However, due to its complex formulations and high computational requirements, the method is time-consuming and struggles with scalability, particularly when applied to large-scale cells. For faster layout generation, several acceleration techniques, such as those in [8], are typically incorporated into the SMT formulation. But such acceleration techniques may have a negative impact on solution quality, as noted in [6]. Furthermore, the work [7] proposes an efficient search-based method to generate CFET placements, carefully considering transistor folding and routing feasibility during the search process. This method excels at achieving multiple high-quality placement solutions. However, since the method does not tightly integrate placement and routing, the resulting placements may not fully ensure in-cell routability. Therefore, it is desirable to develop a more efficient and effective method to synthesize CFET cells that ensure in-cell routability within limited routing resources.

In this paper, we develop a comprehensive placement and routing framework to efficiently synthesize high-quality CFET cells. Our major contributions are summarized as follows:

- For large-scale cells, we propose a breadth-first search (BFS)-based partitioning technique to ensure scalability and efficiency. Additionally, we employ a strategy that combines heuristic sorting with sequential partition processing to mitigate the quality degradation that may be caused by partitioning.
- A novel SMT-based placement method that integrates partial routing is proposed to achieve optimal-width placement while ensuring in-cell routability. Particularly, this method also determines the pin positions for each net, which simplifies subsequent routing complexity and improves overall efficiency.
- Based on the pin positions of each net determined during the placement stage, we propose a progressive metal routing method that includes a manual-inspired M0 routing followed by an ILP-based M1 and M2 routing. This method effectively addresses the challenges of routing resource scarcity and unidirectional routing in CFET technology.
- Compared with the state-of-the-art CFET cell generators [7] and [4], experimental results show that our algorithm achieves an average reduction of 3% in cell width, with 7 out of 30 cells exhibiting smaller widths. For the remaining 23 cells, which have the same cell width as those in [7] and [4], our algorithm achieves the smallest M2 usage and total metal length.

II. PRELIMINARIES

In this section, we first give the problem statement, and then introduce the cell architecture and challenges of CFET.

This work was partially supported by the National Natural Science Foundation of China under Grant 92373207, and the National Key Research and Development Program of China under Grant 2022YFB4400500. (†Corresponding authors: Chao Wang and Ziran Zhu.)

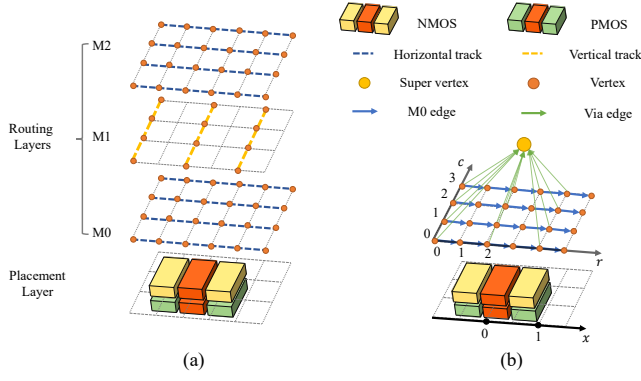


Fig. 2. (a) CFET cell architecture. (b) Modeling example for placement and partial routing.

A. Problem Statement

Given a SPICE netlist, cell architecture, and design rules, the objective of the CFET cell layout generation is to determine optimal transistor placement and routing paths that minimize cell width and overall wire length. Each transistor in the SPICE netlist includes three terminals (i.e., source, gate, and drain) associated with specific nets. Besides, the generated layout must meet the given placement and routing rules. The placement rules mainly include the non-overlap rule, diffusion sharing rule, and diffusion jog rule, while the routing rules encompass the minimum area rule (MAR), end-of-line (EOL), via rule (VR), parallel run length (PRL), step height rule (SHR), and minimum I/O pin length (MPL), consistent with [4, 7], and more details can be found in [4].

B. Cell Architecture and Challenges

The CFET cell architecture used in this paper, aligned with the architecture in [7] and [4], follows a 4.5T structure with layers arranged as placement, M0, M1, and M2 (see Figure 2(a)). The 4.5T structure incorporates four horizontal routing tracks in both the M0 and M2 layers for horizontal routing, while the M1 layer is designated for vertical routing, with vias enabling interlayer connections. Since the distribution of routing resources and specific routing rules, this CFET architecture poses the following two significant challenges to in-cell routability.

M1 routing resources: In the routing process, M1 tracks are required for I/O nets and the nets that cannot fully connect in the M0 layer. However, as shown in Figure 2(a), the vertical tracks on the M1 layer are sparsely arranged, limiting available routing resources. Additionally, the complex routing rules further restrict these resources; for instance, Figure 3(a) shows that a single vertical track cannot accommodate two different nets due to MAR and EOL rules.

M0 routing legalization: The routing results in the M0 layer tend to be more compact than those in the M1 and M2 layers, which presents greater challenges for legalization on the M0 layer. Figure 3(b) illustrates an example of a PRL violation; due to the need to adhere to other rules (such as the EOL rule), adjustments to rectify the issue are often not feasible. In fact, during M0 routing, it is common to encounter situations where adjusting a metal segment to meet one rule might lead to a conflict with another.

III. THE PROPOSED ALGORITHM

Figure 4 summarizes the overall flow of our algorithm, which mainly consists of three stages: (1) preprocessing, (2) SMT-based routability-guaranteed placement, and (3) progressive metal routing with placement-specified pin positions. We shall detail these three major parts in the following subsections.

A. Preprocessing

The preprocessing stage includes three main steps: transistor folding, BFS-based partitioning, and a heuristic strategy for quality maintenance. These steps ensure compliance with the design rule for maximum

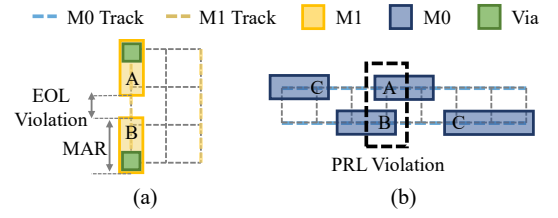


Fig. 3. Applying MAR/EOL/VR/PRL/SHR/MPL = 1/2/1/1/2/2. (a) Example of EOL rule violation on the vertical track of the M1 layer. (b) Example of routing result with PRL violation.

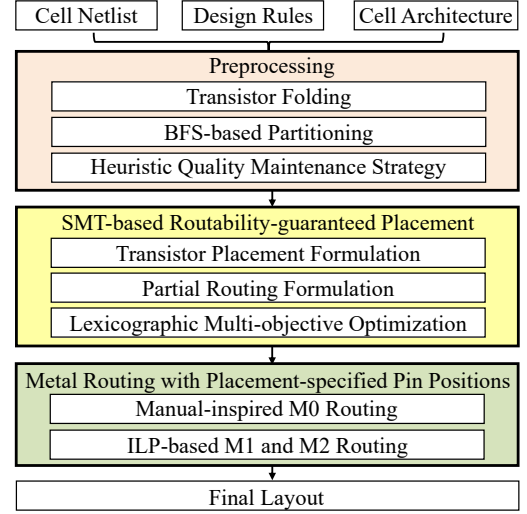


Fig. 4. The overall flow of our algorithm.

transistor width and also accelerate the subsequent placement process with minimal impact on solution quality.

1) *Transistor Folding*: When a transistor's width exceeds the specified maximum, it must be divided into smaller segments, known as "fingers". To reduce the risk of diffusion jog violations, our algorithm strives to maintain the uniform widths of the fingers while minimizing the total number of fingers after folding.

2) *BFS-based Partitioning*: To ensure the scalability and efficiency of the algorithm, a BFS-based partitioning method is employed for cells containing more than 20 transistors. The input netlist is represented as a graph, where each net connected to the active region is treated as a vertex, and each transistor is represented as an edge. For each vertex that connects both NMOS and PMOS transistors, we initiate a BFS search from this vertex, continuing until reaching a VDD or VSS. Transistors visited during a BFS iteration are grouped into the same partition. This method effectively ensures that transistors are systematically grouped based on their potential for diffusion-sharing.

3) *Heuristic Quality Maintenance strategy*: After partitioning, the partitions are organized by considering the following key factors to mitigate the potential impact on the solution quality: (1) Diffusion sharing potential: If a partition cannot share diffusion with both adjacent partitions simultaneously, the final cell width will increase. (2) Crossing nets range: When nets span more widely across partition boundaries, the final routing length tends to increase accordingly.

To find the best arrangement, all possible orderings are explored to maximize diffusion sharing and minimize the range of crossing nets. Considering these factors helps minimize the impact of partitioning on solution quality. After sorting, a partition list is generated and the partitions are sequentially placed in the specified order.

B. SMT-based Routability-Guaranteed Placement

In this section, we propose a novel SMT-based placement method that integrates partial routing to achieve optimal-width placement while en-

suring in-cell routability. Our routability-guaranteed placement model is depicted in Figure 2(b), which preserves the structure of the placement and M0 layers, while the M1 and M2 layers are abstracted as a super vertex. This abstraction significantly reduces the number of variables associated with vertices and edges.

For transistor placement, a one-dimensional x -coordinate system is established on the placement layer, as shown in Figure 2(b). When a transistor terminal needs to connect to the routing layer, four M0 vertices directly above the terminal are designated as potential pin positions. For partial routing, our model includes concrete vertices on the M0 layer and a super vertex that represents the higher routing layers. Besides, two types of directed edges are established: horizontal M0 edges connecting vertices from left to right, and via edges linking M0 vertices to the super vertex. To facilitate the estimation of M2 usage and wire length, a two-dimensional coordinate system is established on the M0 layer, with coordinates designated as r and c , as shown in Figure 2(b). Importantly, it should be noted that when a net connects to the super vertex through a via edge, it represents the net links to the M1 tracks in the actual cell architecture, as illustrated in Figure 5.

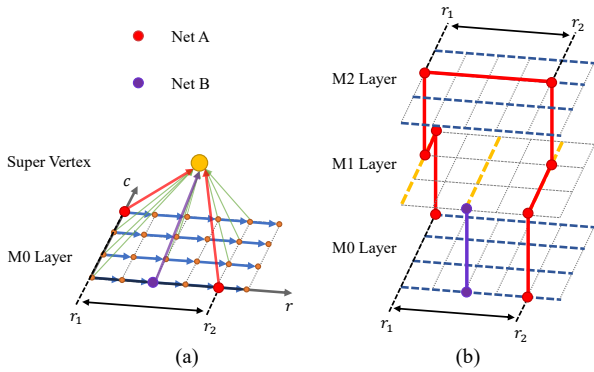


Fig. 5. (a) Example of nets connecting to the super vertex through via edges. (b) Mapping of (a) in the actual cell architecture.

In the following subsections, we shall detail the formulation for our routability-guaranteed placement model. The primary variables and symbols are listed in Table I.

TABLE I
NOTATIONS USED IN OUR FORMULATION.

Symbol	Description
T	The set of transistors.
t_i	The i^{th} transistor.
$Nmos$	The set of NMOS.
$Pmos$	The set of PMOS.
x_i	Unsigned integer variable. Represents the x -coordinate of t_i .
f_i	0/1 variable. Indicates whether transistor t_i is flipped.
N	The set of nets.
n	The n^{th} net.
V	The set of vertices.
V_c	The set of M0 vertices.
v	A vertex.
sv	The super vertex representing the higher layers (i.e., M1 and M2 layers).
E	The set of edges.
$e_{u,v}$	Edge connecting vertices u and v .
E_v	The set of edges connected to the vertex v .
d_e^v	-1/1 constant. Indicates the directional parameter for edge e connected to vertex v . $d_e^v = 1$ if e directs toward v and $d_e^v = -1$ otherwise.
s_n^v	0/1 variable. Indicates whether v is a pin of net n .
v^n	0/1 variable. Indicates whether v is occupied by net n .
f_n^v	Signed integer variable. Represents the total flow of net n for v .
e^n	0/1 variable. Indicates whether e is occupied by net n .
f_e^n	Signed integer variable. Represents the flow of net n through edge e .
p_n^v	0/1 variable. Indicates whether v is used as a patch of net n .
m_r	Unsigned integer variable. Represents the estimated usage of M2 track at row r .
m_r^n	0/1 variable. Indicates whether one of the M2 tracks is used for net n at row r .

1) *Transistor Placement Formulation*: A placement result can be obtained by determining the variables x_i and f_i for each transistor t_i . Additionally, to mitigate the complexity of the subsequent routing problem, the pin positions for each net are defined by introducing the

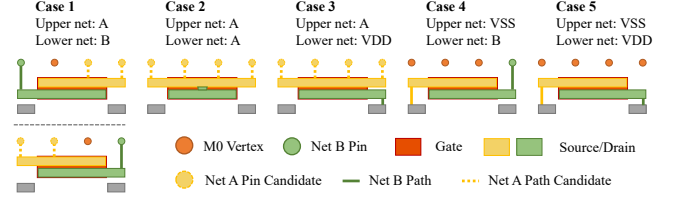


Fig. 6. Five distinct cases for pin selections.

s_n^v variables for each M0 vertex. The constraints associated with the transistor placement rules are outlined as follows, using NMOS as an example; the constraints for PMOS are identical.

Non-overlap constraints: Constraint (1) ensures that transistors do not occupy the same location.

$$\forall t_i, t_j \in Nmos, \bigwedge_{i \neq j} (x_i \neq x_j). \quad (1)$$

Diffusion sharing constraints: For two adjacent transistors, t_i (left) and t_j (right), we define the function $\text{Share}(t_i, t_j, f_i, f_j)$, where f_i and f_j denote the flip types of t_i and t_j , respectively. The function returns 1 if the diffusion region can be shared between t_i and t_j under the flip types f_i and f_j ; otherwise, it returns 0.

$$(x_i + 1 = x_j) \rightarrow (\text{Share}(t_i, t_j, f_i, f_j) = 1). \quad (2)$$

Diffusion jog constraints: For three transistors designated as t_i (left), t_j (middle), and t_k (right), with widths w_i , w_j , and w_k respectively. The following constraint must be satisfied for these transistors:

$$(x_i + 1 = x_j) \wedge (x_j + 1 = x_k) \rightarrow (w_j \geq w_i) \vee (w_j \geq w_k). \quad (3)$$

Pin position allocation: In scenarios where a transistor terminal must be connected through the routing layer, the terminal is extracted to the M0 layer, with the exit vertex designated as the pin for the corresponding net. Taking the active region as a reference, as illustrated in Figure 6, five distinct cases are identified, each requiring individual consideration. Let v_1, v_2, v_3 , and v_4 represent the four candidate pins arranged from left to right on M0, and nets A and B representing the associated nets. In Case 5, extraction to M0 is unnecessary. The constraints for the remaining four cases are formulated as follows:

$$\begin{aligned} \forall n \in \{A, B\}, \quad & (s_n^{v_1} + s_n^{v_2} + s_n^{v_3} + s_n^{v_4}) \leq 1, \\ \text{For case 1: } & ((s_B^{v_1} = 1) \wedge (s_A^{v_3} + s_A^{v_4} = 1)) \vee ((s_B^{v_4} = 1) \wedge (s_A^{v_1} + s_A^{v_2} = 1)), \\ \text{For case 2 and case 3: } & (s_A^{v_1} + s_A^{v_2} + s_A^{v_3} + s_A^{v_4} = 1), \\ \text{For case 4: } & (s_B^{v_4} = 1). \end{aligned} \quad (4)$$

It is important to note that the constraints on pin selection for the gate in CFET follow the same principles as those applied in the active region.

2) *Partial Routing Formulation*: This section first presents a theorem and an accompanying lemma that ensures in-cell routability, followed by the formulation of partial routing.

In-Cell Routability Assurance: As illustrated in Figure 5(b), the M2 routing outcomes are governed by the usage of M1 tracks, which in turn depend on the usage of vias from M0 to M1 and the M0 routing result. Furthermore, based on the CFET architecture and observations from experimental results in [7] and [4], CFET cells generally exhibit minimal use of M2 resources. Accordingly, we have the following assumptions: (A1) The routing resources on the M2 layer are sufficient, and routing legalization on the M2 layer can be ensured. (A2) If a net requires multiple vias from M0 to M1, it will occupy an M2 track spanning the horizontal range between its leftmost and rightmost vias.

Given Assumptions A1 and A2, along with the two challenges discussed in Section II-B, we present the following theorem:

Theorem 1. *For a given placement, in-cell routability is guaranteed if the routing solution on the M0 layer satisfies the following two conditions: (C1) The M0 routing solution is legal. (C2) In this M0*

routing solution, I/O nets and nets that are not fully routed on M0 are linked to M1 tracks through vias. Additionally, the selection of via and M1 tracks must comply with the VR rule, with each M1 track accommodating only one net.

Proof. We categorize the nets into three types based on Conditions C1 and C2:

- 1) Nets fully routed on M0: Since the M0 routing solution is legal (Condition C1), these nets are considered fully routed.
- 2) Nets occupying a single M1 track: These nets include I/O nets that are fully routed on M0, such as net *B* in Figure 5(b). Since each M1 track can accommodate only one net (Condition C2), legalization of the M1 routing is ensured, and these nets are regarded as fully routed.
- 3) Nets spanning multiple M1 tracks: These nets require M2 routing for connectivity. As an example, consider net *A* in Figure 5(b): this net should occupy an M2 track between r_1 and r_2 (Assumption A2), where r_1 and r_2 represent the horizontal coordinates of the leftmost and rightmost vias utilized by the net. Since sufficient M2 routing resources are available and M2 legalization is guaranteed (Assumption A1), complete routing for these nets is ensured.

Hence, by meeting Conditions C1 and C2, all nets achieve complete and legal routing, ensuring in-cell routability. This concludes the proof. $\square$

As shown in Figure 5, the M0 routing and via edges in our model correspond directly to those in the cell architecture. Based on the preceding analysis and Theorem 1, we present the following lemma:

Lemma 2. *The in-cell routability of the placement can be guaranteed by satisfying the following three conditions during the partial routing in our model: (C3) Each pin is connected to the super vertex. (C4) Both the M0 routing and the selection of via edges comply with the design rules. (C5) The usage of M2 tracks, as estimated based on Assumption A2, does not exceed the available resources.*

Proof. Condition C5 reinforces Assumption A1, and by satisfying Conditions C3 and C4 in the model, Conditions C1 and C2 are fulfilled in the cell architecture, thereby ensuring the in-cell routability (Theorem 1). This concludes the proof. $\square$

Note that Condition C3 tightens Condition C2. Our approach does not distinguish whether a net requires the use of higher routing layers, but explicitly mandates that each net must connect to the M1 layer.

Formulation: With notations provided in Table I, the partial routing formulation utilizes s_n^v as input to generate v_n^v and p_n^v for each vertex, and e^n for each edge, where p_n^v plays a pivotal role in the legalization.

To facilitate routing, we adopt the concepts of source, sink, flow, and commodity from the multi-commodity flow (MCF) problem, involving the transport of multiple commodities from sources to sinks. Constraint (5) defines the MCF formulation, with the definitions of f_n^v , f_n^e , E_v and d_e^v provided in Table I.

$$\forall n \in N, v \in V, \quad f_n^v = \sum_{e \in E_v} (f_n^e \times d_e^v). \quad (5)$$

Next, we present Constraint (6), which specifies the determination of e^n and v^n by f_n^e . The function Binary returns 0 if the input number is 0, and 1 otherwise.

$$\begin{aligned} \forall e \in E, n \in N, \quad e^n &= \text{Binary}(f_n^e), \\ \forall v \in V, n \in N, \quad v^n &= \text{Binary}\left(\sum_{e \in E_v} e^n\right). \end{aligned} \quad (6)$$

Constraint (7) defines the sink and sources for each net by determining the value of f_n^v for each vertex. To satisfy condition C3, each pin is designated as a source with a commodity of quantity 1, while the super vertex serves as the shared sink for all nets. This configuration ensures that commodities flow from the pins to the super vertex, thereby

guaranteeing that each pin can reach the super vertex and satisfying Condition C3.

$$\begin{aligned} \forall n \in N, \quad f_n^{sv} &\leq 0, \\ \forall n \in N, v \in V, \quad f_n^v &= s_n^v. \end{aligned} \quad (7)$$

The routing rules are formulated as constraints within the SMT solver, following a similar approach to that outlined in [4], to ensure the legality of M0 routing and via edge selection, thereby satisfying Condition C4.

To enhance the flexibility of legalization, the patch variable p_n^v is introduced. For example, as shown in Figure 7(a), when I/O net *A* has only one pin, it requires a connection solely to the super vertex. However, this configuration would violate the MAR rule, necessitating an extension on M0. Figure 7(b) shows the legalization result without the utilization of p_n^v based on the MCF formulation. In contrast, by incorporating p_n^v , we reduce unnecessary extensions, as illustrated in Figure 7(c).

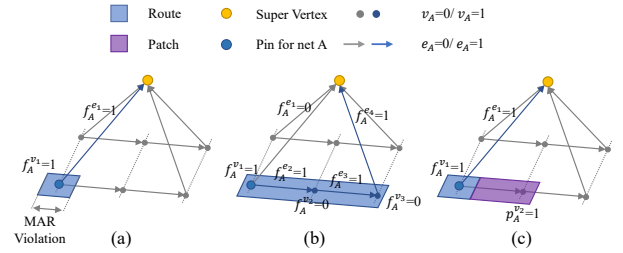


Fig. 7. (a) Partial routing result for net *A* without adherence to routing rules. (b) Legalization for net *A* without the utilization of p_n^v . (c) Legalization for net *A* with the incorporation of p_n^v .

The methodology for calculating the estimated usage of M2 tracks is described in Constraint (8), where LR_n and RR_n denote the leftmost and rightmost horizontal coordinates of the via edges utilized by net *n*. The definitions of m_r and m_r^n are provided in Table I. The function InRange returns 1 when the input *r* falls within the range defined by LR_n and RR_n ; otherwise, it returns 0.

$$\begin{aligned} m_r^n &= \text{InRange}(r, LR_n, RR_n), \\ m_r &= \sum_{n \in N} m_r^n. \end{aligned} \quad (8)$$

Based on Constraint (8), the constraint required to satisfy Condition C5 is derived and presented in Constraint (9), where H_n denotes the horizontal track number.

$$m_r \leq H_n. \quad (9)$$

Thus, it has been ensured that the partial routing solution satisfies Conditions C3, C4, and C5, thereby guaranteeing the in-cell routability (Lemma 2). Our routability-guaranteed placement method significantly reduces the scale of modeling and formulation compared to the approach in [4]. For instance, for a placement with width *k*, the model established in [4] contains $(20k - 8)$ vertices and $(27k - 16)$ edges, while our method results in $(8k - 3)$ vertices and $(12k - 8)$ edges. Furthermore, for a net with *l* pins in the aforementioned placement, the formulation in [4] requires distinguishing between different commodities, leading to $(l - 1) \times (47k - 24)$ flow variables associated with that net. In contrast, our method does not require distinguishing between different commodities for the same net, allowing each vertex and edge to use a single flow variable, resulting in a total of $(20k - 11)$ flow variables.

3) *Lexicographic Multi-Objective Optimization:* The framework incorporates multiple optimization objectives during the placement stage. The primary optimization objective is the cell width, as detailed in Expression (10). X_r represents the horizontal coordinate value of the rightmost transistor.

$$\text{Min.} \quad X_r. \quad (10)$$

If the partitioning method is applied, we modify Expression (10) to Expression (11). C_w represents the width correction term which is assigned -1 if the placement result from the current iteration allows diffusion sharing with the subsequent partition, and 0 otherwise.

$$\text{Min. } X_r + C_w. \quad (11)$$

The second objective is the estimated usage of M2 tracks, as detailed in Expression (12). $RLimit$ represents the upper bound of the horizontal coordinate r , which is determined by the total number of transistors.

$$\text{Min. } \sum_{r \in [0, RLimit]} m_r. \quad (12)$$

The third optimization objective is the half-perimeter wire length (HPWL) defined by the positions of the pins for each net, as detailed in Expression (13). $MaxR_n$, $MinR_n$, $MaxC_n$, and $MinC_n$ denote the values of r and c corresponding to the leftmost, rightmost, topmost, and bottommost pins of net n , respectively.

$$\text{Min. } \sum_{n \in N} (MaxR_n - MinR_n + MaxC_n - MinC_n). \quad (13)$$

These three optimization objectives are integrated into the overall formulation by employing a lexicographic multi-objective optimization approach.

C. Progressive Metal Routing with Placement-Specified Pin Positions

After obtaining the placement result and the pin positions for each net, we perform a progressive metal routing method that includes a manual-inspired M0 routing followed by an ILP-based M1 and M2 routing. Figure 8 illustrates our routing method.

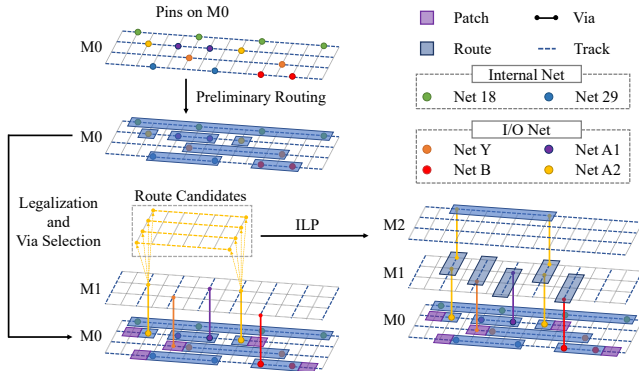


Fig. 8. Illustration of our routing method using the AOI21x1 as an example.

1) *Manual-inspired M0 Routing*: The M0 routing process is inspired by manual routing. Initially, a preliminary routing step is performed for each net, establishing direct connections between its pins along the M0 tracks, provided they are unobstructed by pins from other nets. Then, the M0 routing undergoes a legalization step, where segments requiring extension to comply with routing rules are lengthened using patches. During this step, appropriate via positions are determined for pins that need to connect to M1, with the objective of minimizing the distance between vias when multiple vias are required for a net.

2) *ILP-based M1 and M2 Routing*: After completing the M0 routing, the via positions connecting M0 to M1 for each net are treated as pin positions on M1. For nets with multiple pins on M1, the nets are partitioned into several 2-pin subnets, and each subnet presents four potential routing paths, as depicted in the “Route Candidates” of Figure 8. An ILP formulation is then employed to select an appropriate routing path for each subnet while conducting the legalization process.

IV. EXPERIMENTAL RESULTS

Our algorithm was implemented in Python and executed on a workstation with Intel i7-12700H 2.70GHz CPU and 32GB memory. The SMT solver Z3 [3] (version 4.13.0) was employed to obtain the optimal solution for the SMT-based routability-guaranteed placement formulation presented in Section III-B and the ILP problem detailed in Section III-C2.

Besides, the same as in [4] and [7], we used the SPICE netlists of ASAP 7nm standard cells from [5] as the input netlists for synthesizing CFET cells. To ensure a fair comparison, we followed the same placement and routing rules¹ as those used in [4] and [7], with NMOS stacked on PMOS in the placement.

TABLE II
COMPARISON OF THE CFET CELLS GENERATED BY DAC-2024 [7], TVLSI-2021 [4] AND OUR ALGORITHM. THE BEST RESULTS ARE HIGHLIGHTED IN BLUE BASED ON A PRIORITIZED ORDER OF CELL WIDTH, M2 USAGE, AND METAL LENGTH.

Cell Netlist	Cell Width (#CPP)				M2 Usage			Metal Length			Runtime (sec.)		
Name	#FET	[7]	[4]	Ours	[7]	[4]	Ours	[7]	[4]	Ours	[7]	[4]	Ours
AND2x2	6	6	6	6	0	0	0	56	60	56	0.5	8.1	5.3
AND3x1	8	6	6	6	0	0	0	63	68	63	1.4	30.1	8.8
AND3x2	8	7	7	7	0	0	0	69	76	69	1.6	28.4	10.9
AOI22x1	8	11	11	11	1	1	0	203	240	176	20.5	363.2	131.3
BUFx2	4	5	5	5	0	0	0	39	40	39	0.2	4.9	3.2
BUFx3	4	6	6	6	0	0	0	56	53	52	0.6	11.2	4.2
BUFx4	4	7	7	7	0	0	0	62	59	58	0.8	7.9	5
BUFx8	4	12	12	12	0	0	0	107	105	103	4.4	43.7	11.5
DFHQN	24	15	16	15	1	0	0	203	182	172	410.3	683.8	93
FA	24	14	14	14	3	2	2	276	379	249	484.6	6653.1	191.8
INVx1	2	3	3	3	0	0	0	22	23	22	0.1	0.5	1.3
INVx2	2	4	4	4	0	0	0	27	29	27	0.1	1	1.8
INVx4	2	6	6	6	0	0	0	47	48	46	0.6	3.5	3
INVx8	2	10	10	10	0	0	0	111	92	86	3.4	19.1	6.7
NAND2x1	4	6	6	6	0	0	0	70	74	65	1.3	15.9	5
NAND2x2	4	10	10	10	0	0	0	125	131	118	5.3	33.8	13.8
NAND3x1	6	11	11	11	0	0	0	147	146	142	12.2	124.1	15.2
NOR2x1	4	6	6	6	0	0	0	70	74	66	1.3	12.9	4.7
NOR2x2	4	10	10	10	0	0	0	119	131	118	5	27.9	31.9
NOR3x1	6	11	11	11	0	0	0	163	156	138	11.1	52.3	17.9
OR2x2	6	6	6	6	0	0	0	56	60	56	0.5	13	5.5
OR3x1	8	6	6	6	0	0	0	63	68	63	1.5	76.8	10.3
OR3x2	8	7	7	7	0	0	0	69	76	69	1.7	89.2	13.7
Avg.1 [†]	6.61	8.04	8.09	8.04	0.22	0.13	0.087	96.65	103.04	89.26	42.1	628.4	25.9
Norm.1 [†]		1.00	1.00	1.00	2.44	1.44	1.00	1.08	1.15	1.00	1.63	24.26	1.00
AOI21x1	6	9	9	8	0	0	1	122	142	129	6.3	119.7	19
NAND3x2	6	21	21	20	0	0	1	303	283	308	81.9	2869.5	641.9
NOR3x2	6	21	21	20	1	0	1	320	286	310	82.3	1897.5	483.7
OA121x1	6	9	9	8	0	0	1	131	149	139	7.9	52.5	18.2
OA122x1	8	11	11	10	0	1	2	174	255	208	19.3	612.6	89.2
XNOR2x1	10	11	11	10	1	1	2	180	223	170	47.5	977	58.9
XOR2x1	10	11	11	10	1	1	2	175	213	170	50.4	134.9	84.5
Avg.2 [‡]	6.8	9.27	9.3	9.03	0.27	0.2	0.4	120.93	130.7	116.23	42.15	703.9	66.4
Norm.2 [‡]		1.03	1.03	1.00	0.68	0.5	1.00	1.04	1.12	1.00	0.63	10.6	1.00

[†] Avg.1 and Norm.1 are calculated after excluding the 7 cells that our algorithm achieves smaller widths, while Avg.2 and Norm.2 include all cells.

Table II compares the layouts generated by our algorithm with those produced by the state-of-the-art CFET cell generators, DAC-2024 [7] and TVLSI-2021 [4]. In the table, the column “#FET” represents the number of transistors, and “Cell Width” provides the width of the generated cell. “Metal Length” is calculated as the weighted sum of vias and metal grids, following the method used in [7] and [4], where vias are weighted 4× compared to metal grids to account for parasitic resistance. “M2 Usage” indicates the usage of the M2 tracks, and “Runtime” shows the runtimes in seconds. Additionally, since the codes of [7] and [4] are not available for us, the experimental results of [7] and [4] are quoted from the respective publications directly.

As shown in Table II, the layouts generated by our algorithm achieve an average reduction of 3% in cell width compared to those in [7] and [4]. Notably, our algorithm achieves the smallest cell width for all tested cells, with 7 out of 30 cells exhibiting smaller widths. For the remaining 23 cells, which have the same cell width as those in [7] and [4], our algorithm achieves the best M2 usage and total metal length. Besides, although the runtime in Table II is just for reference due to the differences in machines, our algorithm should still run much faster than [4]. This demonstrates the effectiveness of our routability-guaranteed placement formulation, as well as the efficiency of our progressive metal

¹ MAR/EOL/VR/PRL/SHR/MPL=1/2/1/1/2/2

TABLE III
RE-GENERATED CFET LAYOUTS BY OUR GENERATOR AFTER
ADJUSTING OPTIMIZATION OBJECTIVES.

Cell Netlist	Cell Width			M2 Usage			Metal Length		
	[7]	[4]	Ours	[7]	[4]	Ours	[7]	[4]	Ours
AOI21x1	9	9	9	0	0	0	122	142	118
NAND3x2	21	21	21	0	0	0	303	283	272
NOR3x2	21	21	21	1	0	0	320	286	288
OAI21x1	9	9	9	0	0	0	131	149	127
OAI22x1	11	11	11	0	1	0	174	255	175
XNOR2x1	11	11	11	1	1	1	180	223	176
XOR2x1	11	11	11	1	1	1	175	213	175
Avg.	13.29	13.29	13.29	0.43	0.43	0.29	200.71	221.57	190.14
Norm.	1.00	1.00	1.00	1.48	1.48	1.00	1.05	1.17	1.00

TABLE IV
COMPARISON OF LARGER CFET CELLS GENERATED BY DAC-2024 [7]
AND OUR GENERATOR.

Cell Netlist	#FET	Cell Width		M2 Usage		Metal Length		Runtime(sec.)	
		[7]	Ours	[7]	Ours	[7]	Ours	[7]	Ours
DHLx1	16	11	11	0	0	117	115	441	210.71
DHLx2	16	12	12	0	0	117	117	121	169.17
DHLx3	16	13	13	0	0	146	134	131	136.29
DFHQNx2	24	16	16	1	0	212	174	420	98.46
DFHQNx3	24	17	17	1	0	227	191	502	103.07
SDFHx4	32	N/R	24	N/R	1	N/R	337	N/R	1217.14
SDFLx4	32	N/R	24	N/R	1	N/R	338	N/R	1195.4
Avg. [†]	19.2	13.8	13.8	0.4	0	163.8	146.2	323	143.54
Norm. [†]		1.00	1.00			1.12	1.00	2.25	1.00

[†] The calculations for Avg. and Norm. exclude the two cells, SDFHx4 and SDFLx4, which are not reported (N/R) in [7].

routing with placement-specified pin positions. Although the overall runtime of our algorithm may be higher than that of [7], it remains reasonable and worthwhile given our algorithm's superiority in metrics such as cell width. Furthermore, for the two largest cells, *DFHQN* and *FA*, we employed the partitioning method discussed in Section III-A2, resulting in a runtime shorter than that of [7]. This further illustrates the effectiveness of our preprocessing strategy for managing large circuits.

For the 7 cells that our algorithm achieves smaller widths, the M2 usage and metal length in some layouts produced by our algorithm exceeds those of [7] and [4]. This is due to the fact that the reduced cell widths necessitate additional long-distance M2 tracks to accommodate the smaller cell widths. To validate this perspective, we combined Expression (10) and Expression (12) by applying weighted parameters and re-experimented on these 7 cells, which led to improvements in both metal length and M2 usage compared to [7] and [4], as shown in Table III. Nevertheless, it is important to note that optimizing cell width remains a primary concern in standard cell layout generation. Therefore, compromising other objectives to achieve smaller cell widths is justifiable, provided the trade-off stays within an acceptable range.

To further assess the robustness and scalability of our algorithm on larger cells, we conducted experiments similar to those in [7]. Table IV compares the results, showing that our algorithm produces better layouts with smaller M2 usage and metal length, while maintaining the same cell width and achieving comparable runtime.

Finally, Figure 9 provides a comparison of the layouts generated by [7] and our algorithm before and after adjusting objectives. The layout produced by our algorithm before objective adjustment demonstrates a smaller cell width, while the layout generated after the adjustment shows notable reductions in both M2 usage and metal length. This highlights the flexibility and effectiveness of our algorithm.

V. CONCLUSIONS

In this paper, we have presented a comprehensive placement and routing framework for synthesizing CFET cells. We first presented a partitioning technique followed by a heuristic quality maintenance strategy for large-scale cells to ensure scalability and efficiency. Then, we proposed a novel SMT-based placement method incorporating partial routing to achieve minimum-width placement while ensuring in-cell routability. Finally, we proposed a progressive metal routing method

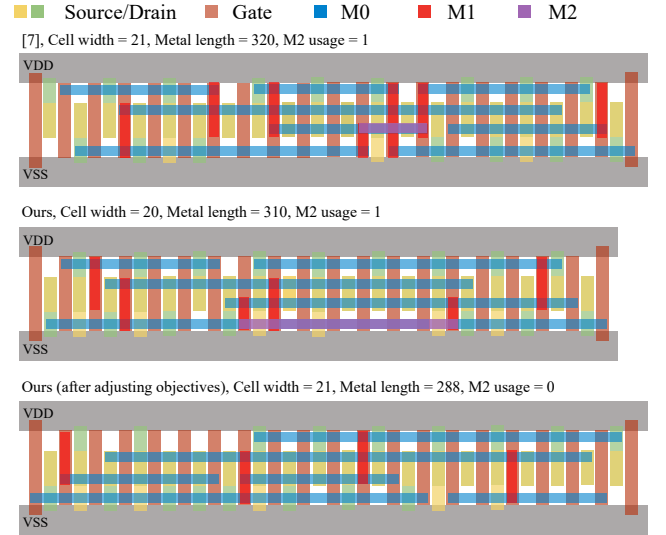


Fig. 9. Comparison of the layout generated by [7], our algorithm and our algorithm after adjusting objectives for netlist NOR3x2.

that includes a manual-inspired M0 routing followed by an ILP-based M1 and M2 routing. Experimental results have shown that our algorithm achieves the smallest cell width for all tested cells, with 7 out of 30 cells exhibiting smaller widths. For the remaining 23 cells, which have the same cell width as those in other generators, our algorithm achieves the smallest M2 usage and total metal length.

REFERENCES

- [1] G. Bae et al. 3nm GAA technology featuring multi-bridge-channel FET for low power and high performance applications. In *2018 IEEE International Electron Devices Meeting (IEDM)*, pages 28.7.1–28.7.4, 2018.
- [2] M. G. Bardon et al. Dimensioning for power and performance under 10nm: The limits of FinFETs scaling. In *2015 International Conference on IC Design & Technology (ICICDT)*, pages 1–4, 2015.
- [3] N. Björner, A.-D. Phan, and L. Fleckenstein. *vz - an optimizing SMT solver*. In C. Baier and C. Tinelli, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 194–199, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [4] C.-K. Cheng et al. Complementary-FET (CFET) standard cell synthesis framework for design and system technology co-optimization using SMT. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 29(6):1178–1191, 2021.
- [5] L. T. Clark et al. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics Journal*, 53:105–115, 2016.
- [6] K. Jo and T. Kim. Optimal transistor placement combined with global in-cell routing in standard cell layout synthesis. In *2021 IEEE 39th International Conference on Computer Design (ICCD)*, pages 517–524, 2021.
- [7] S. Kim and T. Kim. Optimal transistor folding and placement for synthesizing standard cells of complementary FET technology. In *2024 61st ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2024.
- [8] D. Lee et al. SP&R: SMT-based simultaneous place-and-route for standard cell synthesis of advanced nodes. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(10):2142–2155, 2021.
- [9] N. Loubet et al. Stacked nanosheet gate-all-around transistor to enable scaling beyond FinFET. In *2017 Symposium on VLSI Technology*, pages T230–T231, 2017.
- [10] E. Park and T. Song. Complementary FET (CFET) standard cell design for low parasitics and its impact on VLSI prediction at 3-nm process. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 31(2):177–187, 2023.
- [11] D. Prasad et al. Buried power rails and back-side power grids: Arm® CPU power delivery network design beyond 5nm. In *2019 IEEE International Electron Devices Meeting (IEDM)*, pages 19.1.1–19.1.4, 2019.
- [12] J. Ryckaert et al. The complementary FET (CFET) for CMOS scaling beyond n3. In *2018 IEEE Symposium on VLSI Technology*, pages 141–142, 2018.
- [13] S. M. Y. Sherazi et al. CFET standard-cell design down to 3track height for node 3nm and below. In *Advanced Lithography*, 2019.
- [14] S. Subramanian et al. First monolithic integration of 3D complementary FET (CFET) on 300mm wafers. In *2020 IEEE Symposium on VLSI Technology*, pages 1–2, 2020.
- [15] H.-S. P. Wong. Beyond the conventional transistor. *IBM Journal of Research and Development*, 46(2.3):133–168, 2002.